NATIONAL UNIVERSITY OF OSTROH ACADEMY
Department of Information Technologies and Data Analytics

Lab 6.
Responsive layout, container queries, fluid typography, and debugging CSS layout

Modern Web Development: HTML, CSS, JS/TS

1. Aim

Consolidate practical skills in building responsive web interfaces using modern CSS layout mechanisms, and learn to identify and fix responsive layout problems using browser DevTools.

Throughout this lab, you will progressively implement a responsive interface using:

- Flexbox;
- CSS Grid;
- Media queries;
- Subgrid;
- Container queries;
- Fluid typography with `clamp()`;
- CSS logical properties and relative units;
- Browser DevTools for layout analysis.

Particular emphasis is placed not on writing CSS "by template", but on experimenting, observing layout behavior, and justifying each fix.

2. Objectives and learning outcomes

After completing this lab, you must be able to:

- understand the difference between one-dimensional Flexbox and two-dimensional CSS Grid;
- combine Flexbox and Grid in a single interface;
- create a responsive layout without an excessive number of breakpoints;
- understand the purpose of media queries and cases where they are not needed;
- use `repeat()`, `minmax()`, and `auto-fit` / `auto-fill`;

- understand the purpose of subgrid and use it to align nested components;
- understand the difference between media queries and container queries;
- create a component that adapts to the width of its container rather than the viewport;
- use `clamp()` for fluid typography;
- use browser DevTools to analyze Grid/Flexbox, computed styles, the box model, and overflow;
- find the root cause of a responsive layout problem, rather than merely masking its visual symptom;
- critically verify CSS code suggested by an AI agent.

3. Lab tasks

1. Conduct brief research on Flexbox, Grid, media queries, container queries, and fluid typography.
2. Complete a series of code challenges of increasing complexity.
3. Build a responsive layout without using fixed widths where they are not needed.
4. Implement a component that uses a container query.
5. Implement fluid typography without a set of breakpoints for every viewport size.
6. Use browser DevTools to identify and fix deliberately introduced layout problems.
7. Use an AI agent to analyze at least one challenge and independently verify the proposed solution.
8. Prepare an email with deployment links and a conclusion covering experiment results and adopted CSS decisions (§8).

> **Starter project.** Markup for all challenges and the page with deliberately introduced errors for Challenge 8 — layout-lab-starter.zip. A description of its contents is in the `README.md` inside the archive.

4. Research tasks

Before starting the practical part, study the theoretical materials and conduct brief experiments. The experiment results (observations and conclusions) will be needed for the conclusion in your email (§8) and the oral defense (§10). General references — Learn CSS — web.dev and CSS — MDN.

4.1. Flexbox and Grid

Study:

- Layout — web.dev Learn CSS
- Flexible box layout — MDN

- CSS grid layout — MDN

Research:

- What problem does Flexbox solve?
- What problem does CSS Grid solve?
- Why is Flexbox called a one-dimensional layout, and Grid a two-dimensional layout?
- What changes after applying `display: flex`?
- What changes after applying `display: grid`?
- How do `flex-wrap`, `justify-content`, and `align-items` work?
- How do `minmax()`, `repeat()`, and the `fr` unit work?

Mini-experiment

Create a container with six elements and sequentially test:

```
display: flex;
display: grid;
grid-template-columns: repeat(3, 1fr);
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
```

Record how the layout behavior changes as the viewport width varies.

4.2. Media queries

Study:

- Media queries — MDN

Research:

- what `@media` specifically defines;
- how a media query differs from a simple CSS property change;
- which breakpoints are truly necessary;
- whether a problem can be solved using Grid/Flexbox without a media query;
- how to inspect media queries in DevTools.

Experiment

Compare two approaches:

```
@media (width < 768px) {
  .cards {
    grid-template-columns: 1fr;
  }
}
```

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
}
```

Formulate a conclusion: when is a media query genuinely needed, and when can responsive behavior be a built-in property of the layout itself?

4.3. Container queries

Study:

- Container queries — MDN
- CSS containment — MDN

Research:

- how `@container` differs from `@media`;
- what `container-type` is used for;
- what `container-type: inline-size` means;
- how the queried container is determined;
- in which cases container queries make a component more reusable.

Basic experiment

```
.card-wrapper {
  container-type: inline-size;
}

@container (width > 500px) {
  .card {
    /* change layout */
  }
}
```

Change the width of `.card-wrapper` directly, without changing the viewport, and observe the result.

4.4. Fluid typography

Study:

- Adapting typography to user preferences with CSS — web.dev
- clamp() — MDN

Research:

- the difference between `px`, `rem`, and `vw`;
- the purpose of `clamp()`;

- the structure of `clamp(min, preferred, max);`
- why fluid typography can reduce the number of breakpoints.

Experiment

Compare:

```
h1 {
  font-size: 32px;
}

@media (width >= 768px) {
  h1 {
    font-size: 48px;
  }
}
```

```
h1 {
  font-size: clamp(2rem, 5vw, 4rem);
}
```

Resize the viewport and inspect the computed `font-size` in DevTools.

4.5. Subgrid

Study:

- CSS subgrid — web.dev
- Subgrid — MDN

Research:

- what problem subgrid solves;
- how nested grid differs from subgrid;
- how tracks of the parent Grid are inherited;
- in which cases subgrid helps align card contents.

Experiment

Both options use cards inside a shared Grid container:

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 24px;
}
```

Nested Grid — each card defines its own rows:

```
.card {
  display: grid;
  grid-template-rows: auto 1fr auto;
}
```

Subgrid — the card spans three rows of the parent Grid and uses those rows directly:

```
.card {
  display: grid;
  grid-row: span 3;
  grid-template-rows: subgrid;
}
```

Without `grid-row: span 3`, the card spans only a single row of the parent, leaving `subgrid` with nothing to inherit.

Explain: what problem does subgrid solve that is more difficult to handle with an ordinary nested Grid?

5. Practical part: Code challenges

The markup for each challenge is provided in the starter project.

Challenge 1. Flexbox: responsive header

Initial state

```
<header class="header">
  <a class="logo" href="#">WebLab</a>

  <nav class="navigation">
    <a href="#">Home</a>
    <a href="#">Projects</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </nav>

  <button class="header-action">Get started</button>
</header>
```

Tasks:

1. place logo, navigation, and button on a single line;
2. ensure vertical alignment;
3. maintain spacing between elements using `gap`;
4. allow the navigation to wrap onto a new line;

5. do not use `position: absolute` to build the layout;
6. verify behavior on a narrow viewport.

Bonus challenge: make the navigation automatically wrap onto the next line when space is insufficient, without using a media query.

Expected result: the header remains functional across different viewport widths.

Challenge 2. Grid: adaptive card layout

```
<section class="features">
  <article class="feature-card">...</article>
  <article class="feature-card">...</article>
  <article class="feature-card">...</article>
  <article class="feature-card">...</article>
  <article class="feature-card">...</article>
  <article class="feature-card">...</article>
</section>
```

Tasks:

- cards must have a minimum width of 220px;
- number of columns adapts to available space;
- uniform `gap` between cards;
- no horizontal overflow;
- no separate media queries for 2, 3, or 4 columns.

Use: `repeat()`, `minmax()`, `auto-fit`.

Experiment: compare `auto-fit` and `auto-fill`, explaining the difference based on the outcome.

```
repeat(auto-fit, minmax(220px, 1fr))
repeat(auto-fill, minmax(220px, 1fr))
```

Challenge 3. Responsive page layout: Grid + Flexbox

Build a complete layout.

Desktop:

```
+--------------------------------------+
|                Header                |
+-------------+------------------------+
|             |                        |
|   Sidebar   |          Main          |
|             |                        |
+-------------+------------------------+
|                Footer                |
+--------------------------------------+
```

Use Grid for the general page layout and sidebar + main. Use Flexbox for header, navigation, grouping action buttons, and individual components.

On a narrow viewport:

```
+---------------------+
|        Header       |
+---------------------+
|         Main        |
+---------------------+
|       Sidebar       |
+---------------------+
|        Footer       |
+---------------------+
```

Choose the breakpoint yourself or, if possible, implement the layout shift without a media query.

Do not use: `float`; tables for layout; absolute positioning as the primary layout mechanism; a fixed width for the entire page.

Challenge 4. Subgrid: aligned cards

Cards have varying content lengths. Align headings, body text, and "Read more" buttons at identical vertical levels without fixed heights.

First, implement a solution without subgrid. Afterwards, replace the implementation with subgrid (see the experiment in §4.5) and compare the results: your answer to the §4.5 question must draw directly on this challenge.

Challenge 5. Container query: reusable card

```
<div class="content-layout">
  <aside class="sidebar">
    <div class="card-wrapper">
      <article class="product-card">...</article>
    </div>
  </aside>

  <main>
    <div class="card-wrapper">
      <article class="product-card">...</article>
    </div>
  </main>
</div>
```

The same `.product-card` must behave differently depending on container width: vertically in a narrow container, and side-by-side with an image and text block in a wide container.

Condition: do not use `@media` to alter the card layout. Use `container-type` and `@container`.

```
.card-wrapper {
  container-type: inline-size;
}

@container (width > 500px) {
  .product-card {
    display: grid;
    grid-template-columns: auto 1fr;
  }
}
```

`@container` modifies the styles of the container's **descendants**, not the container itself; therefore, `container-type` is assigned to the `.card-wrapper` wrapper rather than `.product-card`.

Experiment: place the same card inside a sidebar, main content area, and a modal-like container; verify that it adapts correctly regardless of the viewport width.

Bonus: explore `container-name` and implement a named container.

```
container: card / inline-size;
```

Challenge 6. Container query units

Continue the previous challenge and make the card's heading size depend on the container width.

Research `cqw`, `cqi`, `cqmin`, and `cqmax`, comparing them with `vw`.

Experiment: place identical cards in a wide container and a narrow container without resizing the viewport. Explain why `vw` and `cqw` produce different behaviors.

Challenge 7. Fluid typography

```
<section class="hero">
  <p class="eyebrow">Modern Web Development</p>
  <h1>Build interfaces that adapt to their environment.</h1>
  <p class="hero-text">
    Responsive interfaces should adapt naturally to available space.
  </p>
  <a class="button" href="#">Explore</a>
</section>
```

Initial CSS uses breakpoint-based typography:

```
.hero h1 { font-size: 32px; }
.hero-text { font-size: 16px; }

@media (width >= 768px) {
  .hero h1 { font-size: 44px; }
  .hero-text { font-size: 18px; }
}

@media (width >= 1200px) {
  .hero h1 { font-size: 56px; }
  .hero-text { font-size: 20px; }
}
```

Replace it with fluid typography using `clamp()`. Choose the exact values yourself.

```
.hero h1 {
  font-size: clamp(...);
}
```

Typography should not become too small, nor excessively large, and must scale smoothly without a separate breakpoint for each font size.

DevTools experiment: resize the viewport, select `h1`, open Computed, check the actual `font-size`, and record at least three values.

Challenge 8. Debug the layout

The starter project (`challenge-08` folder) contains deliberately introduced CSS errors. On a narrow viewport, the page exhibits horizontal scroll, a clipped hero heading, cards overflowing the container, and a sidebar occupying a disproportionate amount of space.

```
.page {
  width: 1200px;
  display: grid;
  grid-template-columns: 300px 1fr;
}

.main {
  width: 1000px;
}

.cards {
  display: flex;
  gap: 24px;
}

.card {
  width: 350px;
}

.hero h1 {
  font-size: 64px;
  white-space: nowrap;
}
```

Tasks:

1. open browser DevTools;
2. identify the element causing overflow;
3. inspect its box model;
4. check computed `width`, `min-width`, and `max-width`;
5. inspect Grid/Flexbox overlay;
6. locate the CSS rule causing the issue;
7. determine the root cause;
8. make a minimal fix;
9. re-check the layout.

For each problem found, record:

Problem	Cause	CSS property	Fix
Horizontal overflow	...	...	...
Heading overflow	...	...	...

Problem	Cause	CSS property	Fix
Card overflow	...	...	...

> **Important!** Do not treat `overflow-x: hidden` as an automatically correct solution. You must explain why the overflow occurred, not simply hide it.

Challenge 9. AI-generated layout review

The AI agent here serves two roles: first it **generates** a solution, then it **reviews** it with the prompt from §7. Between these steps, you verify the code independently to have a baseline to compare against AI recommendations.

Stage 1. Generation

Generate or obtain a CSS solution from an AI agent for the following task:

> Create a responsive dashboard with a header, sidebar, main content area and a grid of cards. It should work on desktop, tablet and mobile.

Save the generated solution without changes — at the end, you will compare it with the final version.

Stage 2. Self-verification

1. inspect the AI-generated implementation in browser DevTools;
2. independently find at least 3 potential problems or redundant choices.

Pay particular attention to excessive media queries, fixed widths, `min-width`, overflow, duplicate CSS, incorrect usage of Flexbox/Grid, and places where `minmax()/auto-fit` or a container query would do the job.

Stage 3. AI code review

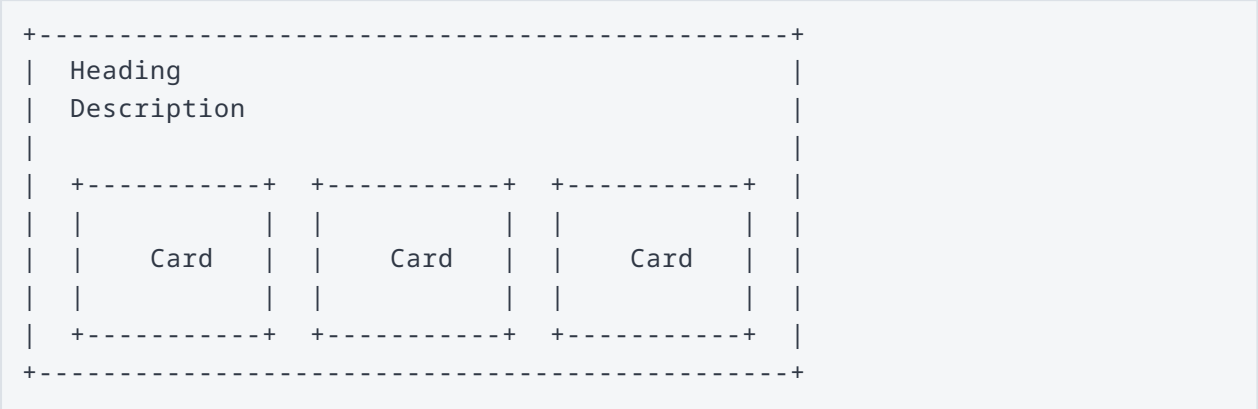
1. pass the generated code to the AI agent with the recommended code review prompt (§7);
2. verify each AI recommendation against MDN/web.dev and compare with your own review findings: what the AI found that you did not, and vice versa;
3. fix justified problems;
4. compare the initial and final CSS.

AI recommendation	Accepted?	Verification	Justification
...	Yes/No	MDN/web.dev	...
...	Yes/No	MDN/web.dev	...
...	Yes/No	MDN/web.dev	...

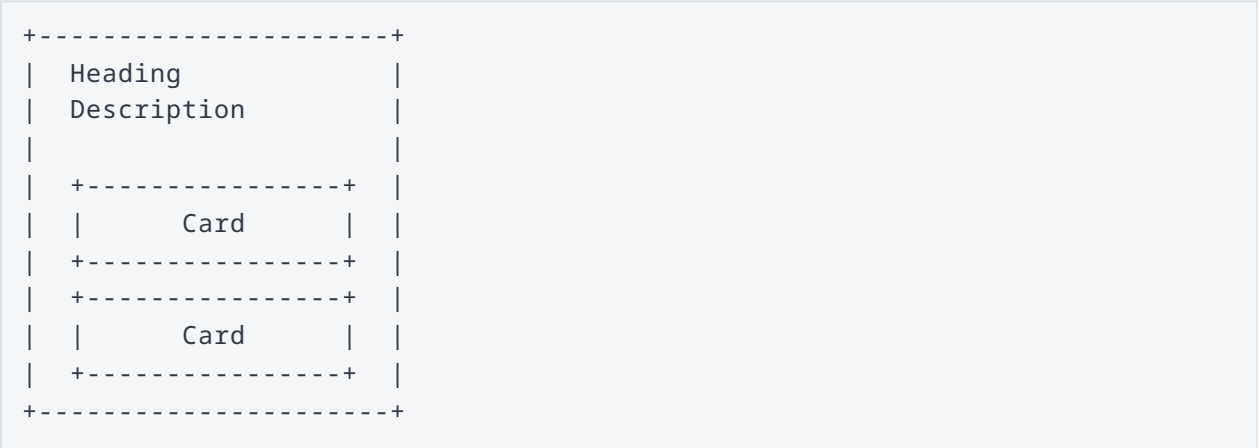
Challenge 10. Final responsive component

Combine acquired skills in a single compact component — a Responsive Feature Section.

Desktop:



Mobile:



Required to use:

- CSS Grid;
- Flexbox for at least one inner component;
- `minmax()` or `auto-fit`;
- `clamp()`;
- Container query for at least one component;
- DevTools to verify responsive behavior.

Optional: subgrid; container query units; named containers.

6. Final implementation requirements

The final implementation must:

1. work correctly on narrow and wide viewports;
2. have no horizontal overflow without a justified reason;
3. contain no fixed width on the main page container;

4. use Grid and Flexbox according to the task;
5. contain no redundant breakpoints;
6. use fluid typography for at least one text category;
7. include at least one container query;
8. not use `overflow-x: hidden` to mask layout issues;
9. not use absolute positioning as the primary layout mechanism;
10. be verified in browser DevTools.

7. AI usage requirements

You may use an AI agent to explain CSS layout, generate an initial solution, identify potential problems, debug, refactor, and compare alternative implementations.

However, you must independently:

- verify the result;
- understand the CSS properties used;
- check recommendations against the documentation;
- explain the rationale for each significant change.

Both prompts below position the AI as a reviewer: it analyzes already-written code without modifying it. The debugging prompt is helpful, among other things, in Challenge 8; the code review prompt is a mandatory step in Challenge 9.

Recommended debugging prompt:

> Analyze the responsive layout of this component. Do not change the code. Find potential problems with overflow, fixed widths, Grid/Flexbox, and media queries. For each problem, explain the root cause and propose a minimal fix.

Recommended code review prompt:

> Review this CSS layout for compliance with modern responsive design practices. Pay attention to Flexbox, Grid, media queries, container queries, fluid typography, overflow, and component reuse. Do not change the code — first produce a list of findings with explanations.

8. Email requirements

Email subject:

```
[Web] Lab 6: Responsive layout, container queries, fluid typography, and debugging CSS layout
```

In the email, include links to:

```
GitHub repository: https://github.com/<username>/<repository>
GitHub Pages: https://<username>.github.io/<repository>/
Vercel: https://<project>.vercel.app/
```

Conclusion:

State what you learned, which layout mechanisms you used, what problem the container query solved, what problem `clamp()` solved, and what you found with DevTools.

9. Grading

A successful defense — **2 points**, OR doing the work in class.

10. Oral defense

Submission includes an oral interview discussing your practical implementation. Even if parts of the CSS were suggested by an AI agent, you must be able to explain every decision, demonstrate it in DevTools, and answer questions with screen sharing.

Sample questions:

1. What is the fundamental difference between Flexbox and CSS Grid?
2. What does one-dimensional layout mean?
3. What are `flex-wrap`, `justify-content`, and `align-items` used for?
4. What does `minmax()` do?
5. How does `auto-fit` differ from `auto-fill`?
6. When is a media query truly necessary?
7. How does `@container` differ from `@media`?
8. What is `container-type` needed for?
9. What is a named container?
10. What do `cqw` and `cqi` units represent?
11. What is `clamp()` used for?
12. Why can `clamp()` reduce the number of breakpoints?
13. What problem does subgrid solve?
14. How does nested Grid differ from subgrid?
15. How does browser DevTools assist in debugging Grid?
16. How does browser DevTools assist in debugging Flexbox?
17. What is horizontal overflow?
18. Why is `overflow-x: hidden` not always a proper fix?
19. How do you identify the element causing overflow?
20. Why do fixed `width`s frequently cause responsive layout issues?

21. How do you decide if a new breakpoint is necessary?
22. How do you verify whether AI suggested a correct CSS solution?
23. Why can a component with a container query be more reusable than one with a media query?
24. Which layout decisions would you keep unchanged after an AI code review, and why?